

Regular Programs

9. Marc's Cakewalk

```
#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <limits.h>
#include <stdbool.h>
void swap(int *a,int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b=temp;
}
int partition(int *x,int start,int end)
{
    int pivot,pindex,i;
    pivot = x[end];
    pindex = start;
    for(i=start;i<end;i++)
    {
        if(x[i]>=pivot)
        {
            swap(&x[i],&x[pindex]);
            pindex = pindex + 1;
        }
    }
    swap(&x[pindex],&x[end]);
    return pindex;
}
void quicksort(int *x,int start,int end)
{
    if(start<end)
    {
        int i = partition(x,start,end);
        quicksort(x,start,i-1);
        quicksort(x,i+1,end);
    }
}
int main(){
    int n, calories_i, *calories;
    int i;
    int sum = 0;
    scanf("%d",&n);
    calories = malloc(sizeof(int) * n);
    for(calories_i = 0; calories_i < n; calories_i++)
    {
        scanf("%d",&calories[calories_i]);
    }
    quicksort(calories,0,n-1);
```

```

for(i=0;i<n;i++)
{
sum += calories[i]*((int)pow(2,i));
}
printf("%d",sum);
// your code goes here
return 0;
}

```

10. Bear and Steady Gene

```

#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
// Function to check if all characters outside the window are balanced (<= n/4)
int is_balanced(int count[], int target) {
if (count['A'] > target || count['C'] > target || count['G'] > target || count['T'] > target) {
return 0; // Not balanced
}
return 1; // Balanced
}
int steadyGene(char* gene, int n) {
int target = n / 4;
int count[128] = {0}; // Using ASCII values as indices for counts
// Initial pass: Count all characters
for (int i = 0; i < n; i++) {
count[gene[i]]++;
}
// If already steady, the required substring length is 0
if (is_balanced(count, target)) {
return 0;
}
int min_len = n;
int left = 0;
// Sliding window: right pointer expands the window
for (int right = 0; right < n; right++) {
// Decrease count of the character entering the window (remaining outside the balanced region)
count[gene[right]]--;
// Inner while loop: left pointer shrinks the window from the left
// as long as the remaining characters outside the window are balanced
while (is_balanced(count, target)) {
// A valid window is found, update the minimum length
// The length of the substring to be replaced is (right - left + 1)
if ((right - left + 1) < min_len) {
min_len = (right - left + 1);
}
// Shrink the window from the left: increase count of the character leaving the window
count[gene[left]]++;
left++;
}
}
return min_len;
}

```

```

int main() {
int n;
// Read the gene length
if (scanf("%d", &n) != 1) return 1;
char* gene = (char*)malloc((n + 1) * sizeof(char));
// Read the gene string
if (scanf("%s", gene) != 1) {
free(gene);
return 1;
}
int result = steadyGene(gene, n);
printf("%d\n", result);
free(gene);
return 0;
}

```

11. Caesar Cipher

```

#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
int main() {
int n,i,j,k;
char ar[101];
unsigned char x;
scanf("%d",&n);
scanf("%s",ar);
scanf("%d",&k);
for(i=0;i<n;i++)
{
x=ar[i];
if(x>=97 && x<=122)
{
x=x+(k%26);
if(x>122)
{
x=96+(x-122);
}
ar[i]=x;
}
else if(x>=65 && x<=90)
{
x=x+(k%26);
if(x>90)
{
x=64+(x-90);
}
ar[i]=x;
}
}
printf("%s",ar);
return 0;
}

```

12. Pangrams

```
#include <stdio.h>
char s[10000];
int main()
{
    gets(s);
    int f[300]={0},ans=0,i;
    int l=strlen(s);
    for(i=0;i<l;i++)
    {
        if(s[i]==' ')
            continue;
        if(s[i]>='a')
            s[i]=s[i]-('a'-'A');
        if(!(f[s[i]]++))
            ans++;
    }
    if(ans!=26)
        printf("not ");
    printf("pangram\n");
    return 0;
}
```

13. Prefix Sum Array

Populate a Prefix-Sum Array from an array of integer elements. Also create a function to calculate the sum of elements between a given range using the Prefix-Sum Array.

```
#include<stdio.h>
void display(int arr[], int n)
{
    int i;
    for (i=0;i<n;i++)
    {
        printf("\t %d ",arr[i]);
    }
}

create_prefix_sum_array(int arr[], int n)
{
    int i;
    for (i = 1; i<n; i++)
    {
        arr[i] = arr[i] + arr[i-1];
    }
}

void sum(int arr[], int n, int a, int b){
    int result;
    if (a == 0)
        result = arr[b];
    else
        result = arr[b] - arr[a-1];
    printf("The sum is %d", result);
}
```

```

}
int main() {
int n, i, arr[10], a, b;
printf("enter the number of elements\n");
scanf("%d",&n);
printf("enter the array elements\n");
for (i=0;i<n;i++)
{
scanf("%d",&arr[i]);
}
printf("\n Original array");
display(arr, n);
create_prefix_sum_array(arr, n);
printf("\n Prefix sum array");
display(arr,n);
printf("\n enter the range to find the sum");
scanf("%d%d",&a,&b);
sum(arr, n, a, b);
return 0;
}

```

14. Fenwick Tree Construction

Populate a Fenwick Tree from an array of integer elements. Given the value of a node in a Fenwick Tree

$$\text{tree}[k] = \text{sum}(k - p(k) + 1, k)$$

Where, $p(k) = k \& -k$ and denotes the largest power of two that divides k

```

#include<stdio.h>
#include<math.h>
int sum(int arr[], int n, int a, int b){
int i, sum=0;
for (i = a; i<=b; i++)
{
sum += arr[i] ;
}
return sum;
}
void create_fenwick_tree(int T[], int arr[], int n)
{
int a, b, k;
for (k=1; k<=n; k++)
{
a = k - (k&-k) + 1;
b = k;
T[k] = sum(arr,n,a,b);
}
}
void display(int arr[], int n)
{

```

```

int i;
for (i=1;i<=n;i++)
{
printf("\t %d ",arr[i]);
}
}
int main() {
int n, i, arr[100], T[100] , a, b;
// printf("the power of 6 is %u \n", largestPowerOf2(7)) ;
printf("enter the number of elements\n");
scanf("%d",&n);
printf("enter the array elements\n");
for (i=1;i<=n;i++)
{
scanf("%d",&arr[i]);
}
printf("\n Original array");
display(arr, n);
create_fenwick_tree(T, arr, n);
printf("\n Fenwick Tree");
display(T,n);
return 0;
}

```

15. Fenwick Tree - Sum of Elements

Given a Fenwick Tree (Binary indexed Tree) with the value of at node k calculated as
 $tree[k] = sumq(k - p(k) + 1, k)$

Where, $p(k) = k \& -k$ and denotes the largest power of two that divides k. Create a function to calculate sum of first 'k' elements in the Fenwick Tree.

```

#include<stdio.h>
#include<math.h>
int sum(int T[], int k) {
int s = 0;
while (k >= 1) {
s += T[k];
k -= k&-k;
}
printf("\n The sum is %d",s) ;
}
void display(int arr[], int n)
{
int i;
for (i=1;i<=n;i++)
{
printf("\t %d ",arr[i]);
}
}
int main(){

```

```

int n, k, T[100] = {0, 1, 4, 4, 16, 6, 7, 4, 29};
printf("\n Fenwick Tree");
display(T,n);
printf("\n Enter the value for k");
scanf("%d",&k);
sum(T, k);
return 0;
}

```

Additional Programs

7. Greedy Florist

```

#include <stdio.h>
#include <stdlib.h>
int a[200],i,j,k,l,sum,n;
int com(const void *xx, const void *yy)
{
    if(*(int *)xx > *(int*)yy) return -1;
    return 1;
}
int main()
{
    scanf("%d %d\n",&n,&k);
    for(i=0;i<n;i++)
        scanf("%d",a+i);
    qsort(a,n,sizeof(a[0]),com); // sorts in descending order of price
    sum = 0;
    for(i=0;i<n;i++)
        sum+= a[i]*(1+i/k);
    printf("%d\n",sum);
    return 0;
}

```

8. Camelcase

```

#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <limits.h>
#include <stdbool.h>
int main() {
    char* s = (char *)malloc(10240 * sizeof(char));
    scanf("%s",s);
    int count=0,i;
    for(i=0;i<strlen(s);i++)
    {
        if(s[i]>=65 && s[i]<=90)

```

```

{
count++;
}
}
printf("%d\n",count+1);
return 0;
}

```

9. Mars Exploration

```

#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <limits.h>
#include <stdbool.h>
int main() {
char* S = (char *)malloc(10240 * sizeof(char));
scanf("%s",S);
int i;
int count=0;
for(i=0;S[i]!='\0';i+=3){
if(S[i]!='S'){
count++;
}
if(S[i+1]!='O'){
count++;
}
if(S[i+2]!='S'){
count++;
}
}
printf("%d",count);
return 0;
}

```

10. Funny String

```

#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
int main() {
int T, N, funny, i, r;
char S[10001];
scanf("%d\n", &T);
while (T--) {
scanf("%s\n", S);
N = strlen(S);

```

```

funny = 1;
for(i=1, r=N-2; i<N; i++, r--) {
if (fabs(S[i]-S[i-1]) != fabs(S[r]-S[r+1])) {
funny = 0;
break;
}
}
if (funny)
{
printf("funny");
}
else
printf("not funny");
}
return 0;
}

```

11. Alternating Characters

```

#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
int main()
{
int t;
long i=0;
unsigned int count=0;
char * c;
scanf("%d",&t);
c=(char *)malloc(sizeof(char)*(100002));
while(t--)
{
scanf("%s",c);
for(i=0; c[i] != '\0'; i++)
{
if(c[i]==c[i+1])
{
count++;
}
}
printf("%u\n",count);
count=0;
}
return 0;
}

```

12. Beautiful Binary String

```

#include <math.h>

```

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <limits.h>
#include <stdbool.h>
int main() {
    int n;
    scanf("%d",&n);
    char* B = (char *)malloc(10240 * sizeof(char));
    scanf("%s",B);
    int i=0,count=0;
    while(B[i]){
        if(B[i]=='0'&&B[i+1]=='1'&&B[i+2]=='0'){
            B[i+2]='1';
            count++;
        }
        i++;
    }
    printf("%d",count);
    return 0;
}

```

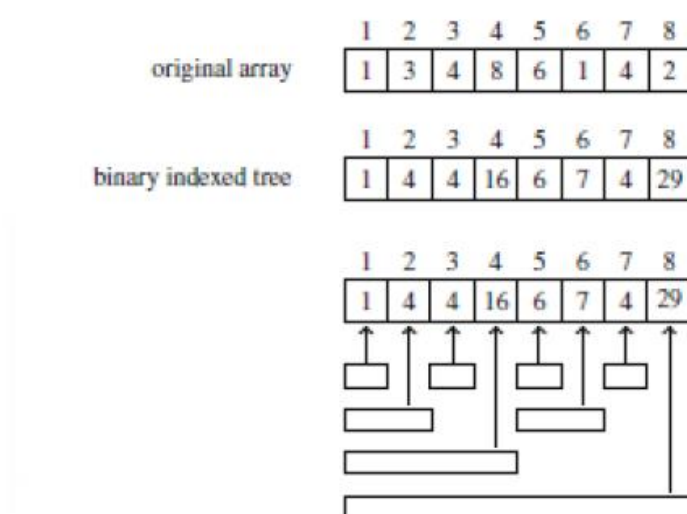
Fenwick Tree - Sum of Elements

Given a Fenwick Tree (Binary indexed Tree) with the value of at node k calculated as

$$\text{tree}[k] = \text{sum}_q (k - p(k) + 1, k)$$

Where, $p(k) = k \& -k$ and denotes the largest power of two that divides k.

Create a function to calculate sum of first 'k' elements in the Fenwick Tree.



```

#include<stdio.h>
#include<math.h>
void display(int arr[], int n)
{
    int i;
    for (i=1;i<=n;i++)
    {
        printf("\t %d ",arr[i]);
    }
}

void add(int T[],int n, int k, int x) {
    while (k <= n) {
        T[k] += x;
        k += k&-k;
    }
    display(T,n);
}

int main(){
    int n=8, k,x, T[100] = {0, 1, 4, 4, 16, 6, 7, 4, 29};
    printf("\n Fenwick Tree");
    display(T,n);
    printf("\n Enter the position and value to be increased ");
    scanf("%d%d",&k,&x);
    add(T,n, k, x);
    return 0;
}

```